Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Ad hoc sıkıştırma yöntemleri
  - Ad hoc yöntemler bir kişiye, bir veriye ya da bir probleme özgü olarak ortaya atılmış yöntemlerdir. Genel amaçlı değildir; günümüzde tercih edilen yöntemler de değildir, ama sıkıştırmanın nereden nereye geldiğini göstermek için incelenir.
- Sparse string sıkıştırma (ekrandan, 00:10:00 ve 00:20:00)
  - Kelimeler arasında birden fazla boşluk bulunan metinler için kullanılır. Bütün boşlukları tek boşluğa indiren bir ön işlem orijinale kayıpsız dönmeyi engeller; bu yöntem boşluk sayılarını korur.
  - Metindeki her karaktere 0, her boşluğa 1 verilir ve bitler 4'lü bloklara ayrılır (L1). Karakterler 0 ile gösterildiği için dizi çok sıfırlıdır, yöntemin adı da buradan gelir.
  - Bir üst seviyede (L2) her 4'lü blok için bir bit yazılır: blok sıfırdan farklıysa 1, tamamen sıfırsa 0. L3 de L2'den aynı kuralla elde edilir. Kısaltılacak bir şey kalmayıncaya kadar seviye eklenir.
  - Örnek metin `Bu__dersin__adı___veri_sıkıştırma___ve_pazartesi_günü__yapılıyor` (_ boşluk):
    - `L1 = 0011|0000|0011|0001|1100|0010|0000|0000|0111|0010|0000|0000|1000|0110|0000|000`
    - `L2 = 1011|1100|1100|1100`, `L3 = 1111`
  - Sıkıştırmada L1 ve L2'deki sıfır bloklar silinir: `L1 = 0011|0011|0001|1100|0010|0111|0010|1000|0` (9 blok, 36 bit), L2 değişmez (16 bit), L3 4 bittir. Çıktıya L1, L2, L3 ve ardından boşluksuz metin yazılır.
  - Başa bir başlık (header) konur: ilk bayt seviye sayısı (3), sonraki baytlar her seviyenin bit sayısı (36, 16, 4).
  - Açma işlemi tersten yapılır (simetrik algoritma): L3'teki her 1, L2'deki sıradaki bloğu alır; L2'deki her 1, L1'deki sıradaki bloğu alır, her 0 yerine `0000` yazılır. Ortaya çıkan L1'de 0 görülen yere metinden sıradaki karakter, 1 görülen yere boşluk konur.
- Baudot kodlaması (ekrandan, 00:30:00)
  - Her sembole 5 bitlik kod verilir. Aynı 5 bitlik kod hem bir harfe (Letters) hem bir rakama/işarete (Figures) karşılık gelir; örneğin `10000` hem A hem 1'dir.
  - Özel kodlar: LS (Letter Shift, `00001`), FS (Figure Shift, `00010`), CR (Carriage Return, `11000`), LF (Line Feed, `10001`), ER (Error, `00011`), na (Not Assigned, `00000`).
  - LS'den sonra gelen kodlar harf, FS'den sonra gelen kodlar rakam/işaret olarak okunur; böylece harf ve rakam karışmaz. Örnek `AB46`: LS `00001`, A `10000`, B `00110`, FS `00010`, 4 `10100`, 6 `10010`.
  - Atanmamış (na) kodlar başka amaçlarla kullanılabilir; hoca örneğin birinin "büyük harfe geç" anlamında kullanılabileceğini söyledi.
- Önek sıkıştırma (front compression) (ekrandan, 00:32:30)
  - Alfabetik sıralı bir sözlük ya da yazım kılavuzu için kullanılır. Her kelime, bir üstteki kelimeyle soldan itibaren ortak olan karakter sayısı ve kalan ek ile yazılır.
  - Örnekler: `aardvark → 1ardvark`, `abaft → 3ft`, `abated → 3ted` (6 karakter 4 karakterle), `abbreviate → 9e` (10 bayt 2 baytla), `abbreviation → 9ion`.
  - Sözlüğün başında az, ortak öneki uzun kelimelerde çok kazanç sağlar. Çözmek için üstteki kelimelere gidilir; hızlı çözmek için bağlı liste mantığı kullanılabilir.
- Run Length Encoding (RLE) (ekrandan, 00:40:00 ve 00:41:40)
  - Bir `d` verisi arka arkaya `n` kez geliyorsa `nd` çiftiyle gösterilir; `n`'ye run denir. Örnek: `111 → 3 1`.

  - Hem metinde hem görüntüde kullanılır; kayıpsız ve kayıplı versiyonları vardır. Run Length Coding, Run Length Compression ya da RLE diye anılır.
  - Tek başına nadiren kullanılır; JPEG gibi formatlarda ön işlemlerden (DCT gibi) sonra ara adım olarak yer alır.
  - Örnek 1 (ekrandan, 00:43:40): `aaaaabbbbbbbbbbbbccccddddddddddeeeeeeeeee` (40 bayt) → `5a12b4c9d10e`. Run'lar birer bayt sayıldığında 10 bayt olur.
  - Örnek 2: `aabccdeefghijjjklmnopqrrstttuvvwwxyyyz` (38 bayt) için RLE çıktısı girdiden uzundur (negatif sıkıştırma); tek ve çift tekrarlar için kazanç yoktur.
- RLE'de run ile veriyi ayırt etme (ekrandan, 00:50:00)
  - `2. all is too well` metni `2. a2l is t2o we2l` olarak kodlanırsa açıcı baştaki 2'nin metne mi ait, run mı olduğunu bilemez (`..` diye açabilir).
  - Çözüm: run'dan önce özel bir escape sembolü (derste `@`) konur. Bu sembol metinde geçebileceği için metinde olmayan bir ASCII karakter seçilir ve hangi karakterin seçildiği başlığa yazılır (örneğin ilk bayt 0 ise escape yok, 1 ise ikinci bayt escape kodu).
  - Run + karakter 2 bayt tutar: `1a` 1 baytı 2 bayta çıkarır, `2a` kazanç sağlamaz, `3a` 3 baytı 2 bayta indirir. Bu yüzden bir varyasyonda yalnız 3 ve daha fazla tekrar run'la yazılır, daha kısa tekrarlar olduğu gibi yazılır; başka versiyonlar en az 4 tekrar ister. Escape sembolü de sayılırsa run'lı yazım 3 bayttır ve kazanç 4 tekrardan itibaren başlar.
- RLE'nin temel problemleri (ekrandan, 01:00:00)
  - Doğal dildeki metinlerde arka arkaya tekrar eden karakter (boşluklar hariç) nadirdir.
  - Run'a 1 bayt ayrıldığı için en büyük değer 255'tir. 260 tane `a` için 255 ve 5'lik iki run yazılır. Run'ın 3 tekrardan sonra başladığı versiyonda 255 değeri 258 karakter anlamına gelir; 258 aynı karakter 3 baytla kodlanır, kalan 2 `a` olduğu gibi yazılır.
  - Escape kodu metinde geçebilir. Exe dosyalarında bütün ASCII karakterler bulunabileceği için farklı bir yol gerekir.
- MNP5 (Microcom Network Protocol) yöntemi (ekrandan, 01:02:10)
  - 3 karakter arka arkaya tekrar ediyorsa çıktıya 3 kopya yazılır, dördüncü bayt run değeridir; escape sembolüne gerek kalmaz.
  - `aaa → aaa0` (3 yerine 4 bayt, genişleme), `aaaa → aaa1` (4 yerine 4, sıkıştırma yok), `aaaaa → aaa2` (5 yerine 4, sıkıştırma var).
- RLE ile MNP5'in karşılaştırılması (ekrandan, 01:06:50)
  - N: karakter sayısı, M: tekrar eden grup sayısı, L: gruplardaki ortalama eleman sayısı.
  - RLE: $\frac{N - ML + 3M}{N}$, MNP5: $\frac{N - ML + 4M}{N}$.
  - $N = 1000$, $M = 10$, $L = 5$ için RLE $0{,}98$ (%2 sıkıştı), MNP5 $0{,}99$ (%1 sıkıştı). Veri değişince sonuç değişebilir; hangisinin her zaman daha iyi olduğu söylenemez.
- Görüntüde RLE: piksel ve renk derinliği
  - Siyah-beyaz resimde bir piksel 1 bit, k bitlik grayscale'de 2^k ton, color table (renk tablosu) resimde 256 renk ve piksel başına 8 bit, true color'da R, G, B için 8'er bit, toplam 24 bit ve 2^{24} renk vardır.
  - Göz bu kadar rengi ayırt edemediği için RLE'den önce nicemleme (quantization, renk sayısını azaltma) yapılır; örneğin (50, 50, 49) ve (50, 50, 48) tek renge indirilir.
- Bitmap'in taranması (ekrandan, 01:40:00)
  - Pikseller scanline'lara (satırlara) yerleşir; ilk piksel sol üst, son piksel sağ alt köşedir. Taramanın başladığı yer ve yön renklerin sürekliliğini etkiler.
  - Tarama satır satır (row-row), sütun sütun (column-column), rotate'li (satır sonunda geri dönerek) ya da zigzag yapılabilir. Rotate'li okumada bir satırın sonundaki renk bir alt satırın başıyla birleşebildiği için run'lar uzar.
- Siyah-beyaz resimde RLE kuralları (ekrandan, 01:40:00)
  - Kompresör ilk pikseli beyaz kabul eder; ilk piksel beyaz değilse çıktıya önce 0 adet beyaz yazılır. Renk kodu yazılmaz, yalnız run'lar yazılır: 17 beyaz, 1 siyah, 55 beyaz → `17 1 55`.
  - Çıktının başına bitmap'in çözünürlüğü (satır ve sütun sayısı) yazılır.
  - Sıkıştırılmış çıktının büyüklüğü şeklin karmaşıklığına bağlıdır; detay arttıkça sıkıştırma kötüleşir.
  - Slayta göre uniform bir alanda RLE için sıkıştırma oranı, şeklin çevre uzunluğunun yarısının alandaki toplam piksel sayısına oranıdır. Her scanline'ın bir giriş, bir çıkış

noktası olduğundan $2\times$ scanline sayısı şeklin çevre uzunluğunu verir.
- 16 renkli resim örnekleri (ekrandan, 01:53:00, 01:58:50 ve 02:00:00)
  - 15 × 11 piksel, 16 renklik palet. Sıkıştırmasız: $15 \cdot 11 \cdot 4 = 660$ bit.
  - 5 renkli şekil satır satır RLE ile 134 sayı verir: $134 \cdot 4 = 536$ bit. Renkler 3'e indirilince (kayıplı) 86 sayı: $86 \cdot 4 = 344$ bit.
  - Çok karışık, her pikselde renk değişen resimde 311 sayı: $311 \cdot 4 = 1244$ bit; çıktı girdiden büyüktür.
  - Run'lar da 4 bitle sayılmıştır, çünkü bu resimde en büyük run 15'tir. Run için illa 1 bayt ayrılması gerekmez; run uzunluğuna göre bit sayısı seçilir ve bu bilgi başlığa yazılır. Gereğinden fazla bit kullanmak artıklık (redundancy) oluşturur.
- Grayscale resimde run ile renk kodunu ayırt etme (ekrandan, 02:07:10 ve 02:09:10)
  - Girdi: `12` × 9, `35 76 112 67`, `87` × 3, `5` × 6, `1`. 1 olan run'lar yazılmazsa açıcı bir sayının run mı renk mi olduğunu bilemez.
  - Yöntem I: Resim 128 grayscale ile sınırlıysa (0-127, 7 bit), her baytın en soldaki biti sayının renk kodu (0) mu run (1) mu olduğunu gösterir. Resim 256 grayscale ise 255 değeri 254'e indirilir ve 255 "arkasından run gelir" işareti olur: `255 9 12 35 76 112 67 255 3 87 255 6 5 1 ...`.
  - Yöntem II: Her 8 değer için önce bir bayt bayrak (flag) gönderilir; bitleri hangi değerin run (1) hangisinin renk (0) olduğunu gösterir: `10000010 9 12 35 76 112 67 3 87 100... 6 5 1 ...`. Her 8 bayt için fazladan 1 bayt gönderilir.
- True color resimde RLE (ekrandan, 02:10:00)
  - Her piksel 3 bayttır. RLE her palet (R, G, B) için ayrı ayrı uygulanır: (171 85 34) (172 85 35) (172 85 30) (173 85 33) → R: `171 172 172 173`, G: `85 85 85 85`, B: `34 35 30 33`.
- Tarama yönünün seçimi ve zigzag (ekrandan, 02:20:00)
  - Dikey çubuklardan oluşan resmi satır satır, yatay çizgilerden oluşan resmi sütun sütun okumak kötü sonuç verir; seçim resmin içeriğine göre yapılır.
  - Çok nesneli, karışık resimlerde resim 8 × 8 (JPEG), 16 × 16, 32 × 32 gibi bloklara bölünür ve her blok zigzag okunur; komşu pikseller genelde aynı ya da yakın renktedir. Boyut blok boyunun katı değilse kenardaki piksel değerleriyle dolgu (padding) yapılır.
  - Örnek 8 × 8 siyah-beyaz resim: row-row `1 1 1 2 1 1 1 <eoln>` × 4 = 32, `7 1 <eoln>` × 2 = 6, `0 8 <eoln>` × 2 = 6, toplam 44 bayt. Column-column için slayt da 44 bayt yazıyor (düzeltme aşağıda).
- Kayıplı RLE (ekrandan, 02:24:30)
  - Daha iyi oran için bazı ihmaller yapılır. Kısa run'lar ihmal edilir: `64 64 64 63 64 64 ...` dizisindeki tek `63`, komşularındaki `64`'e çevrilir.
  - Kullanıcıya ihmal edilebilecek en büyük run uzunluğu sorulur. Kullanıcı 3 derse 1, 2 ve 3 uzunluğundaki run'ların yerine en yakın komşu pikselin değeri konur.

Hocanın Özellikle Vurguladığı Kısımlar

- Kayıpsız dönüş şarttır
  - Bir sıkıştırma yöntemi sıkıştırdığını aynı şekilde açamıyorsa sıkıştırma yöntemi değildir. Sparse string'de boşlukları tek boşluğa indirmek, RLE'de run ile metindeki sayıyı karıştırmak bu yüzden kabul edilmez.
- Simetrik yapı
  - Sparse string ve RLE'de açma, sıkıştırmada yapılanın tersidir. Hangi versiyon kullanıldıysa açıcı da ona göre yazılır; kodlayan ve açan aynı kişi olduğunda serbestlik vardır.
- RLE tek başına çoğu zaman yetersizdir
  - Metinde ardışık tekrar nadir olduğu için çıktı büyüyebilir. En iyi sonuç büyük siyah ve beyaz bölgelerden oluşan resimlerde alınır. Renk sayısı azaldıkça run'lar uzar.
- Başlık (header) bilgisi
  - Seviye sayısı ve bit uzunlukları (sparse string), escape karakteri, run'ların ve renk kodlarının bit uzunluğu, bitmap çözünürlüğü başlığa yazılır; açıcı bunları oradan okur.
- Gereğinden fazla bit artıklıktır
  - Run'a ve renge her zaman 1 bayt vermek redundancy oluşturur; bit sayısı verinin aralığına göre seçilmelidir.

- Her veriye uyan tek yöntem yoktur
  - Verinin içeriği, kaybın kabul edilip edilmediği, sıkıştırma ve açma hızı gibi ölçütlere göre yöntem seçilir. Tarama yönü de resmin içeriğine göre seçilir.

Kısa Tekrar Notları

- Ad hoc: probleme özgü yöntem.
- Sparse string: karakter 0, boşluk 1; 4'lü bloklar; sıfır olmayan blok 1. L1, L2'deki sıfır bloklar silinir. Başlık: seviye sayısı + her seviyenin bit sayısı.
- Baudot: 5 bit; LS `00001`, FS `00010`; aynı kod harf ve rakam olabilir.
- Front compression: üst kelimeyle ortak önek uzunluğu + kalan ek (`abbreviate → 9e`).
- RLE: `nd` çifti, `n` = run. Metinde zayıf, siyah-beyaz resimde güçlü.
- Escape (`@`) + run + karakter = 3 bayt; en az 3 tekrar.
- Run 1 bayt → en fazla 255.
- MNP5: 3 kopya + run bayt; `aaaaa → aaa2`.
- RLE oranı $(N - ML + 3M)/N$, MNP5 $(N - ML + 4M)/N$.
- Siyah-beyaz RLE: ilk piksel beyaz kabul edilir; değilse 0 yazılır; yalnız run'lar yazılır.
- 15 × 11, 16 renk: 660 bit; 536, 344, 1244 bit örnekleri.
- Grayscale: 7 bitte en soldaki bit bayrak; 8 bitte 255 işaret; ya da her 8 değer için 1 bayrak baytı.
- True color: R, G, B ayrı ayrı RLE.
- Zigzag: 8 × 8 bloklar (JPEG); padding kenar pikselleriyle.
- Kayıplı RLE: kısa run'lar komşu renge çevrilir; eşik kullanıcıya sorulur.

Detaylı Açıklamalar

- **Sparse string'de açma.** Hoca açmanın en dikkat edilecek kısım olduğunu söyledi. Başlıktan seviye sayısı (3) ve bit sayıları (36, 16, 4) okunur; bit dizisi L1, L2, L3 olarak bölünür. L3 = `1111` dört bitin de 1 olması nedeniyle L2'nin dört bloğunun hepsini alır (`1011`, `1100`, `1100`, `1100`). L2'nin her biti L1'e gider: `1011` için birinci L1 bloğu `0011`, sonra `0000`, sonra `0011`, sonra `0001`; `1100` için `1100`, `0010`, `0000`, `0000` ve devamı. Böylece silinen sıfır bloklar yerine geri konur ve 64 bitlik orijinal L1 elde edilir. Bir öğrenci silinen sıfır blokların karakter bilgisini kaybettirip kaybettirmediğini sordu. Hoca, karakterlerin çıktıda arka arkaya yazılı olduğunu, sıfırların yalnız karakterlerin nereye geleceğini gösterdiğini, L2 oluşturulurken sıfır blokların yerinin korunduğunu söyledi.
  - Ders sonrası not: Slayttaki boşluksuz metin `budersiadıverisıkıştırmavepazartesigünüyapılıyor` olarak yazılmış, "dersin"in `n`'si eksik. Hoca açarken bunu fark etti (00:20:46). Doğrusu `budersinadıverisıkıştırmavepazartesigünüyapılıyor`'dur; L1'deki ilk 6 sıfırlık karakter grubu "dersin"e karşılık gelir.
- **Baudot'da harf ve rakam karışması.** `AB46` LS ve FS olmadan `10000 00110 10100 10010` diye yazılsaydı açıcı `10000` için tabloda iki karşılık (A ve 1) görür, hangisi olduğunu bilemezdi. LS ve FS kodları bundan sonraki kodların hangi sütundan okunacağını belirtir.
  - Ders sonrası not: Hoca CR'yi "bir satır atladık", LF'yi "satır başı" diye andı. Doğrusu tersidir: Carriage Return satır başına dönmek, Line Feed bir satır aşağı geçmektir.
- **Önek sıkıştırma tablosu.** Her satırdaki sayı, bir üst satırdaki kelimenin soldan kaç karakterinin aynen alınacağını söyler. `abated → 3ted` 6 karakteri 4 karakterle, `abbreviate → 9e` 10 baytı 2 baytla yazar. Kelimeler arasında ortaklık yoksa (hocanın örneği "ay" ardından "bay") kazanç olmaz.
  - Ders sonrası not: Slayttaki tabloda iki satır bir üstteki kelimeyle uyuşmuyor (python ile kontrol edildi): `abandoning` bir üstteki `abandon` ile 7 karakter ortak olduğu için `7ing` olmalıdır (slaytta `3ndoning`); `abate` bir üstteki `abated`'in ilk 5 karakteri olduğu için yalnız `5` yazılabilir (slaytta `3te`). Ayrıca `abandonment` alfabetik sırada `abasement`'tan önce gelir.
- **Negatif sıkıştırma örneği.** İkinci örnekte çoğu karakter bir ya da iki kez geçer. İki `a` zaten 2 bayttır, `2a` da 2 bayttır; tek `b` 1 bayttır, `1b` 2 bayt olur. Bu yüzden çıktı büyür.
  - Ders sonrası not: Slayttaki sıkıştırılmış dizi `...1f1g1i3j...` diye yazılmış, `1h` eksik. 38 karakterlik girdinin doğru RLE çıktısı `2a1b2c1d2e1f1g1h1i3j1k1l1m1n1o1p1q2r1s3t1u2v2w1x3y1z` olup 52 karakterdir (python ile kontrol edildi).
- **Run uzunluğunun bit sayısı.** 134 sayının hepsinin 4 ile çarpılması ilk bakışta hatalı

görünür, çünkü run için 1 bayt denmişti. Hoca bu resimde en büyük run'ın 15 olduğunu, 4 bitin yettiğini, bu yüzden run'lar için de 4 bit kullanıldığını açıkladı. Run'lar 64'ten küçükse 6 bit, 128'den küçükse 7 bit yeter; bu bilgi başlığa yazılır, açıcı ilk 4 biti run, sonraki 4 biti renk kodu olarak okur.
  - Ders sonrası not: 8 bit 0-255 aralığını tutar. 128-255 arası run'lar için 8 bit gerekir; 256 ve üzeri bir run tek bir 8 bitlik alana sığmaz, ya daha fazla bit ayrılır ya da run bölünür. Altyazıdaki "256'nın üzeriyse 8 bayt" ifadesi büyük olasılıkla "8 bit" demek istemiştir.
- **Uniform alan formülü.** Hoca slayttaki "yarısı" kelimesinden önce şüphe edip "iki katı" olabileceğini düşündü, sonra kaynak kitaba bakıp "yarısıymış" dedi. Hoca pratikte bu oranın hesaplanmayacağını, resmin tamamı için çıktı/girdi oranına bakılacağını söyledi.
- **Siyah-beyaz 8 × 8 örneği (ekrandan, 02:20:00).** Satır satır okumada dört satır `1 1 1 2 1 1 1` + eoln (8 bayt), iki satır `7 1` + eoln (3 bayt), iki satır `0 8` + eoln (3 bayt): $32 + 6 + 6 = 44$ bayt. Sütun sütun okumada üç sütun `5 1 1 1` + eoln, dört sütun `0 4 1 1 1 1` + eoln, bir sütun `4 4` + eoln.
  - Ders sonrası not: Slaytta sütun sütun toplamı da 44 bayt yazıyor ve hoca "aynı" dedi. Slayttaki satırlarla toplam $5 \cdot 3 + 7 \cdot 4 + 3 \cdot 1 = 15 + 28 + 3 = 46$ bayttır. Hoca rotate'li okumanın bu örnekte bazı run'ları birleştireceğini (örneğin satır sonundaki siyahın alt satırın siyahlarıyla birleşmesi) söyledi.
- **RLE'nin kullanıldığı yerler.** Hoca RLE'nin JPEG'in ara adımlarında (DCT'den sonra), WinRAR ve ZIP içinde, MNP5 ile modem protokollerinde ve fakslarda kullanıldığını söyledi.
  - Ders sonrası not: ZIP'in standart yöntemi Deflate (LZ77 + Huffman) büyük olasılıkla ayrı bir RLE adımı içermez; ayrı RLE adımı bzip2 gibi araçlarda bulunur.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.